

Processes and Threads

Questions and answers

Question 1

In Fig. 2-2, three process states are shown: **running**, **blocked**, and **ready**. In theory, with three states, there could be six transitions, two out of each state. However, only four transitions are shown.

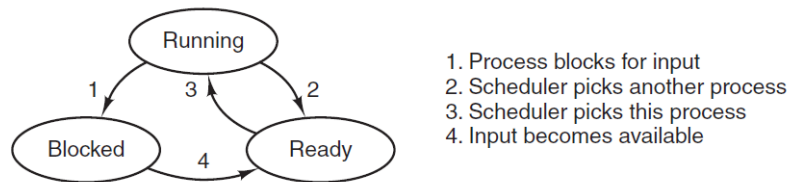


Figure 2-2. A process can be in running, blocked, or ready state. Transitions between these states are as shown.

Are there any circumstances in which either or both of the missing transitions might occur?

Answer 1

Yes, one of the missing transitions can occur, but the other cannot.

- Blocked → Running is possible in a special case. Suppose a process is blocked waiting for I/O and the I/O completes. If the CPU is idle at that moment, the process could go directly from blocked to running.
- Ready → Blocked is impossible. A ready process is not executing, so it cannot request I/O, wait for an event, or do anything else that would block it.

The key rule is: only a **running** process can become blocked.

Question 2

Suppose that you were to design an advanced computer architecture that did process switching in hardware, instead of having interrupts. What information would the CPU need? Describe how the hardware process switching might work.

Answer 2

The CPU would need enough information to save the current process and start another one directly.

- It could have a register containing a pointer to the current process-table entry.
- When I/O completed, the CPU would store the current machine state in the current process-table entry.
- The CPU would then go to the interrupt vector for the device that completed.

- From that vector, it would fetch a pointer to another process-table entry, such as the service procedure or next process to run.
- The CPU would load that process's saved state and start it.

In short, hardware process switching would require the CPU to save the old process state, find the next process-table entry, load that process state, and continue execution from there.

Question 3

On all current computers, at least part of the interrupt handlers are written in assembly language. Why?

Answer 3

Generally, high-level languages do not allow the kind of access to CPU hardware that is required.

- An interrupt handler may need to enable or disable interrupt servicing for a particular device.
- It may need to manipulate data inside a process's stack area.
- Interrupt service routines must execute as rapidly as possible, and assembly language gives very direct control over the generated instructions.

For these reasons, at least the low-level part of an interrupt handler is usually written in assembly language.

Question 4

When an interrupt or a system call transfers control to the operating system, a kernel stack area separate from the stack of the interrupted process is generally used. Why?

Answer 4

There are several reasons for using a separate stack for the kernel.

- The operating system should not crash just because a poorly written user program did not reserve enough stack space.
- Kernel stack data should not be left in a user program's memory space after returning from a system call. Otherwise, a malicious user might use that data to learn information about other processes.

Thus, a separate kernel stack improves both reliability and security.

Question 5

A computer system has enough room to hold five programs in its main memory. These programs are idle waiting for I/O half the time. What fraction of the CPU time is wasted?

Answer 5

Use the CPU utilization formula for multiprogramming:

$$\text{CPU utilization} = 1 - p^n$$

Here, p is the fraction of time a process is waiting for I/O, and n is the number of programs in memory.

In this problem:

$$p = \frac{1}{2}, \quad n = 5$$

The CPU is wasted only when all five programs are waiting for I/O at the same time. Since each program is idle half the time, the probability that all five are idle is:

$$p^n = \left(\frac{1}{2}\right)^5 = \frac{1}{32}$$

So the fraction of CPU time wasted is:

$$\boxed{\frac{1}{32}}$$

This is why the answer is $\frac{1}{32}$: if even one of the five programs is ready to run, the CPU can be used.

Question 6

A computer has 4 GB of RAM, of which the operating system occupies 512 MB. The processes are all 256 MB, for simplicity, and have the same characteristics. If the goal is 99% CPU utilization, what is the maximum I/O wait that can be tolerated?

Answer 6

First, find how many processes can fit in memory.

$$4 \text{ GB} = 4096 \text{ MB}$$

$$\text{Available memory} = 4096 - 512 = 3584 \text{ MB}$$

$$n = \frac{3584}{256} = 14$$

So there is enough room for 14 processes in memory.

Using the CPU utilization formula:

$$\text{CPU utilization} = 1 - p^n$$

For 99% CPU utilization:

$$0.99 = 1 - p^{14}$$

$$p^{14} = 0.01$$

$$p = (0.01)^{1/14} \approx 0.72$$

So the maximum I/O wait that can be tolerated is:

$$\boxed{72\%}$$

This means each process may spend up to about 72% of its time waiting for I/O while still keeping CPU utilization near 99%.

Question 7

Multiple jobs can run in parallel and finish faster than if they had run sequentially. Suppose that two jobs, each needing 20 minutes of CPU time, start simultaneously. How long will the last one take to complete if they run sequentially? How long if they run in parallel? Assume 50% I/O wait.

Answer 7

Each job needs 20 minutes of actual CPU time. Since each job has 50% I/O wait, a job running by itself uses the CPU only half the time.

$$\text{Elapsed time for one job} = \frac{20}{1 - 0.5} = 40 \text{ minutes}$$

If the two jobs run sequentially, the first job finishes after 40 minutes and the second job finishes after another 40 minutes.

$$\text{Sequential completion time} = 40 + 40 = \boxed{80 \text{ minutes}}$$

If the two jobs run in parallel, use the CPU utilization formula:

$$\text{CPU utilization} = 1 - p^n$$

Here:

$$p = 0.5, \quad n = 2$$

$$\text{CPU utilization} = 1 - (0.5)^2 = 1 - 0.25 = 0.75$$

So the CPU is busy 75% of the time. With two similar jobs sharing the CPU, each job gets half of that CPU capacity:

$$\text{CPU share per job} = \frac{0.75}{2} = 0.375$$

Each job therefore receives 0.375 CPU minute per minute of real time. To accumulate 20 minutes of CPU time:

$$\text{Parallel completion time} = \frac{20}{0.375} \approx 53.33 \text{ minutes}$$

So the last job completes after:

$$\boxed{53.33 \text{ minutes}}$$

Thus, sequential execution takes 80 minutes, while parallel execution takes about 53.33 minutes.

Question 8

Consider a multiprogrammed system with degree of 6, that is, six programs in memory at the same time. Assume that each process spends 40% of its time waiting for I/O. What will be the CPU utilization?

Answer 8

Use the CPU utilization formula:

$$\text{CPU utilization} = 1 - p^n$$

Here:

$$p = 0.4, \quad n = 6$$

The CPU is idle only when all six processes are waiting for I/O at the same time:

$$p^n = (0.4)^6 = 0.004096$$

Therefore:

$$\text{CPU utilization} = 1 - 0.004096 = 0.995904$$

So the CPU utilization is:

0.995904 or 99.5904%

Question 9

Assume that you are trying to download a large 2-GB file from the Internet. The file is available from a set of mirror servers, each of which can deliver a subset of the file's bytes; assume that a given request specifies the starting and ending bytes of the file. Explain how you might use threads to improve the download time.

Answer 9

The client process can create multiple threads, with each thread downloading a different byte range of the file from one of the mirror servers.

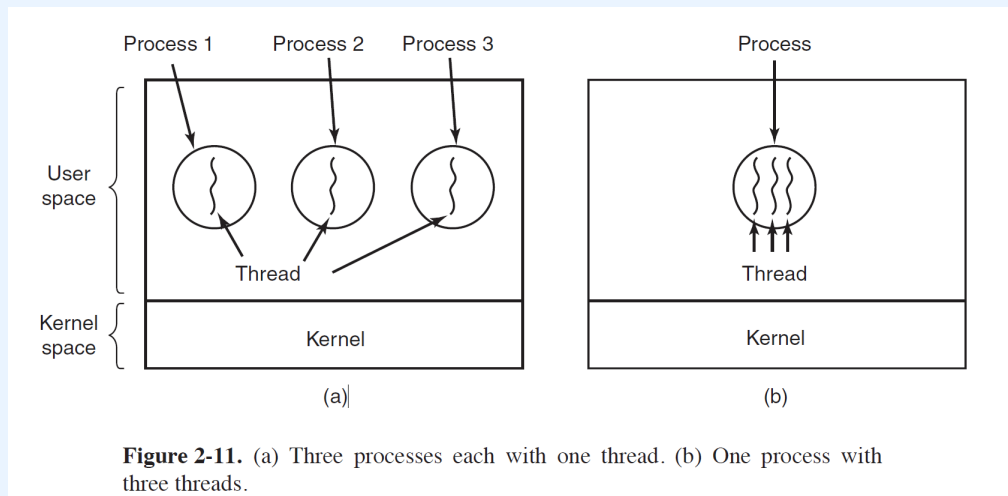
- One thread might download bytes 0 to 499 MB.
- Another thread might download bytes 500 MB to 999 MB.
- Other threads can download the remaining parts in parallel.

After all threads finish, the client combines the downloaded pieces in the correct order to form the complete 2-GB file.

This can improve download time because waiting for one server or one request does not stop the other parts from downloading. However, all threads still share the same network link. If too many threads are used, the network link can become the bottleneck, so adding more threads will no longer improve performance much.

Question 10

In the text it was stated that the model of Fig. 2-11(a) was not suited to a file server using a cache in memory. Why not? Could each process have its own cache?



Answer 10

The model in Fig. 2-11(a) uses separate processes. If each server process has its own memory cache, it becomes difficult to keep the file system consistent.

Suppose one client sends a request to server process 1 to update a file. Server process 1 updates its own cache entry. Shortly after that, another client sends a request to server process 2 to read the same file. If server process 2 also has a cached copy, it may return obsolete data because its cache has not seen the update made by server process 1.

Each process could have its own cache, but then the system would need extra work to keep all caches consistent.

- Server process 1 could write every update through to disk immediately.
- Server process 2 could check the disk on every read to see whether its cached copy is still up-to-date.

This would make the system correct, but it would also cause many disk accesses. That defeats the main purpose of using a cache, which is to avoid slow disk access whenever possible.

Question 11

If a multithreaded process forks, a problem occurs if the child gets copies of all the parent's threads. Suppose that one of the original threads was waiting for keyboard input. Now two threads are waiting for keyboard input, one in each process. Does this problem ever occur in single-threaded processes?

Answer 11

No. In a single-threaded process, there is only one thread of control.

If that single thread is blocked waiting for keyboard input, it is not running and therefore cannot call `fork`. Since it cannot fork while blocked, the situation with two copied threads

both waiting for keyboard input does not occur.

Question 12

In Fig. 2-8, a multithreaded Web server is shown. If the only way to read from a file is the normal blocking `read` system call, do you think user-level threads or kernel-level threads are being used for the Web server? Why?

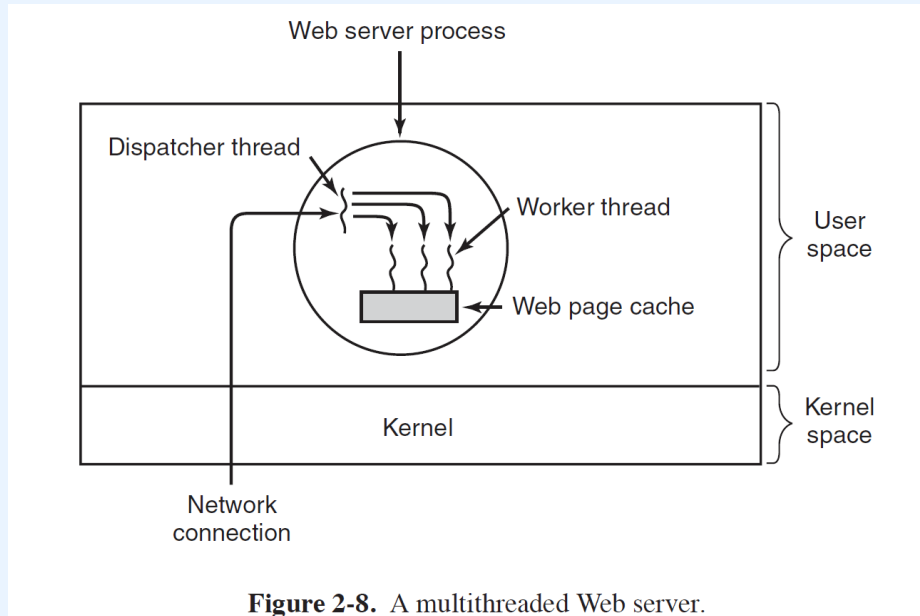


Figure 2-8. A multithreaded Web server.

Answer 12

Kernel-level threads should be used.

A worker thread may have to read a Web page from disk. If the system provides only the normal blocking `read` system call, that worker thread can block while waiting for the disk operation to finish.

If user-level threads are used, the kernel does not know about the individual threads. It sees only one process. Therefore, when one user-level thread makes a blocking system call, the entire process may block. This would stop the dispatcher and the other worker threads too, destroying the benefit of multithreading.

With kernel-level threads, the kernel knows about each thread separately. One worker thread can block on disk I/O while the other threads continue handling requests.

Thus, a Web server using blocking file reads needs kernel-level threads so that one blocked thread does not block the whole server process.

Question 13

In the text, we described a multithreaded Web server, showing why it is better than a single-threaded server and a finite-state machine server. Are there any circumstances in which a single-threaded server might be better? Give an example.

Answer 13

Yes. A single-threaded server can be better when the server is entirely CPU-bound and does not spend much time waiting for I/O.

In that case, multiple threads do not hide any waiting time. They mainly add extra complexity, such as synchronization, scheduling overhead, and possible race conditions. For example, consider a telephone directory assistance server for an area with 1 million people. If each (name, telephone number) record is about 64 bytes, the whole database needs:

$$1,000,000 \times 64 = 64,000,000 \text{ bytes} \approx 64 \text{ MB}$$

This can easily fit in memory. Since each lookup can be answered directly from memory, the server does not block on disk I/O. A simple single-threaded server may therefore be faster and easier to manage than a multithreaded one.

Question 14

In Fig. 2-12 the register set is listed as a per-thread rather than a per-process item. Why? After all, the machine has only one set of registers.

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Figure 2-12. The first column lists some items shared by all threads in a process. The second one lists some items private to each thread.

Answer 14

The physical CPU has only one active set of registers at a time, but each thread needs its own saved register values.

When a thread is running, the CPU registers contain that thread's current state, such as temporary values, stack pointer, and program counter-related information. When the thread is stopped, these register values must be saved so the thread can later continue exactly where it left off.

This is the same idea as multiprogramming processes. When a process is stopped, its register values are saved. When a thread is stopped, its register values must also be saved. Therefore, each thread needs its own register save area. That is why registers are listed as a per-thread item rather than a per-process item.

Question 15

Why would a thread ever voluntarily give up the CPU by calling `thread_yield`? After all, since there is no periodic clock interrupt, it may never get the CPU back.

Answer 15

Threads in the same process usually cooperate with one another. They are normally written as parts of the same application, often by the same programmer, so they are not expected to behave like hostile independent programs.

If yielding helps the application as a whole, a thread may voluntarily give up the CPU. For example, it might yield after finishing a useful unit of work so that another thread in the same process can run.

The important idea is that user-level thread scheduling depends on cooperation. A thread yields because the program's correctness or responsiveness may require it, even though the thread package cannot force another user-level scheduling decision with a periodic clock interrupt.

Question 16

Can a thread ever be preempted by a clock interrupt? If so, under what circumstances? If not, why not?

Answer 16

It depends on whether the threads are user-level threads or kernel-level threads.

- **User-level threads** cannot normally be preempted individually by a clock interrupt, because the kernel does not know about the separate user-level threads. The kernel sees only the process. A clock interrupt can preempt the whole process when its quantum is used up, but the user-level thread package is responsible for choosing which user-level thread runs when the process resumes.
- **Kernel-level threads** can be preempted individually. Since the kernel knows about each thread, a clock interrupt can stop the currently running thread, and the kernel scheduler can choose another thread to run next.

Thus, user-level threads are scheduled mainly by the user-level thread library, while kernel-level threads can be directly scheduled and preempted by the operating system.

Question 17

In this problem you are to compare reading a file using a single-threaded file server and a multithreaded server. It takes 12 msec to get a request for work, dispatch it, and do the rest of the necessary processing, assuming that the data needed are in the block cache. If a disk operation is needed, as is the case one-third of the time, an additional 75 msec is required, during which time the thread sleeps. How many requests/sec can the server handle if it is single threaded? If it is multithreaded?

Answer 17

For a single-threaded server, cache hits take 12 msec. Cache misses take the normal 12 msec plus an additional 75 msec for disk I/O:

$$12 + 75 = 87 \text{ msec}$$

Since two-thirds of the requests are cache hits and one-third are cache misses, the average request time is:

$$\frac{2}{3} \times 12 + \frac{1}{3} \times 87 = 8 + 29 = 37 \text{ msec}$$

So the single-threaded server can handle:

$$\frac{1000}{37} \approx 27.03$$

about 27 requests/sec

For a multithreaded server, disk waiting can be overlapped with useful work by other threads. The effective processing time per request is therefore the 12 msec of non-disk work:

$$\frac{1000}{12} \approx 83.33$$

$83\frac{1}{3}$ requests/sec

Question 18

What is the biggest advantage of implementing threads in user space? What is the biggest disadvantage?

Answer 18

The biggest advantage is efficiency. Switching between user-level threads does not require a trap into the kernel, so thread creation, scheduling, and context switching can be very fast.

The biggest disadvantage is blocking. If one user-level thread makes a blocking system call, the kernel may block the entire process because it does not know about the individual user-level threads. In that case, all threads in the process stop running, even if some of them could otherwise continue.

Question 19

In Fig. 2-15, this program creates ten threads:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *thread_id)
{
    long tid = (long) thread_id;

    printf("Hello World. Greetings from thread %ld\n", tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;

    for (i = 0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %d\n", i);
```

```

        status = pthread_create(&threads[i], NULL,
                                print_hello_world, (void *) (long) i);
    if (status != 0) {
        printf("Oops. pthread_create returned error code %d\n", status);
        exit(-1);
    }
}

pthread_exit(NULL);
}

```

The thread creations and messages printed by the threads are interleaved at random. Is there a way to force the order to be strictly: thread 1 created, thread 1 prints message, thread 1 exits, thread 2 created, thread 2 prints message, thread 2 exits, and so on? If so, how? If not, why not?

Answer 19

Yes. The main program can wait for each thread to finish before creating the next one. After each successful call to `pthread_create`, the main thread should call `pthread_join` for the thread it just created.

```

for (i = 0; i < NUMBER_OF_THREADS; i++) {
    printf("Main here. Creating thread %d\n", i);

    status = pthread_create(&threads[i], NULL,
                            print_hello_world, (void *) (long) i);
    if (status != 0) {
        printf("Oops. pthread_create returned error code %d\n", status);
        exit(-1);
    }

    pthread_join(threads[i], NULL);
}

```

This makes the main thread block until thread i has printed its message and exited. Only then does the loop continue and create thread $i + 1$. So the loop becomes logically:

1. Create thread i .
2. Wait for thread i using `pthread_join`.
3. Create the next thread.

This forces the output order, but it also removes most of the parallelism because only one worker thread runs at a time.

Question 20

In the discussion on global variables in threads, we used a procedure `create_global` to allocate storage for a pointer to the variable, rather than the variable itself. Is this essential, or could the procedures work with the values themselves just as well?

Answer 20

The pointers are really necessary because the size and type of the global variable are not known in advance.

A global variable could be very small, such as a single character, or much larger, such as an array of floating-point numbers. If the thread package stored the value itself, then `create_global` would need to know how many bytes to allocate for that value.

That still leaves a type problem:

- What type should the value parameter of `set_global` have?
- What type should `read_global` return?

Using a pointer avoids these problems. The thread package stores only a pointer, and the application code remains responsible for knowing what type of data that pointer refers to.

Question 21

Consider a system in which threads are implemented entirely in user space, with the run-time system getting a clock interrupt once a second. Suppose that a clock interrupt occurs while some thread is executing in the run-time system. What problem might occur? Can you suggest a way to solve it?

Answer 21

A problem can occur if the clock interrupt happens while the run-time system is manipulating its internal scheduling data structures.

For example, the run-time system might be in the middle of blocking or unblocking a thread and updating the ready queue. At that exact moment, the scheduling queues may be temporarily inconsistent. If the clock interrupt handler inspects those queues and tries to switch threads, it may see invalid state and make a wrong scheduling decision.

One solution is to use a flag:

- When the run-time system starts manipulating thread data structures, it sets a flag saying that it is inside a critical section.
- If the clock handler runs and sees this flag, it does not inspect the queues immediately. Instead, it sets another flag saying that a clock interrupt is pending, then returns.
- When the run-time system finishes its critical work, it checks the pending-clock flag. If the flag is set, it safely runs the clock-handling logic at that time.

This delays the thread switch until the run-time system's queues are consistent again.

Question 22

Suppose that an operating system does not have anything like the `select` system call to see in advance if it is safe to read from a file, pipe, or device, but it does allow alarm clocks to be set that interrupt blocked system calls. Is it possible to implement a threads package in user space under these conditions? Discuss.

Answer 22

Yes, it is possible, but it is inefficient and unattractive.

A user-level thread that wants to make a system call that might block can do the following:

1. Set an alarm timer.
2. Make the potentially blocking system call.
3. If the call returns normally, cancel the alarm timer.

If the system call blocks, the alarm eventually goes off and interrupts the blocked system call, returning control to the user-level threads package. The threads package can then schedule another user-level thread.

The problem is overhead. Most system calls will not block, so each possibly blocking operation becomes three system calls: set the timer, do the operation, and clear the timer. There is also a correctness risk if timers fire too early or too late.

Thus, this approach can work, but it is not a good way to build an efficient threads package.

Question 23

Does the busy waiting solution using the `turn` variable in Fig. 2-23 work when the two processes are running on a shared-memory multiprocessor, that is, two CPUs sharing a common memory?

```
/* Process 0 */
while (TRUE) {
    while (turn != 0)
        ; /* busy wait */
    critical_region();
    turn = 1;
    noncritical_region();
}

/* Process 1 */
while (TRUE) {
    while (turn != 1)
        ; /* busy wait */
    critical_region();
    turn = 0;
    noncritical_region();
}
```

Answer 23

Yes. It still works on a shared-memory multiprocessor, assuming both CPUs see the same shared `turn` variable consistently.

Only the process whose `turn` matches the value of `turn` can enter its critical region. When it leaves, it gives the `turn` to the other process. Thus, both processes cannot be in the critical region at the same time.

However, it is still a busy waiting solution. A process that is not allowed to enter the critical region keeps testing `turn` repeatedly, wasting CPU time.

Question 24

Does Peterson's solution to the mutual-exclusion problem shown in Fig. 2-24 work when process scheduling is preemptive? How about when it is nonpreemptive?

```
#define FALSE 0
#define TRUE 1
#define N 2          /* number of processes */

int turn;             /* whose turn is it? */
int interested[N];    /* all values initially FALSE */

void enter_region(int process)
{
    int other;

    other = 1 - process;
    interested[process] = TRUE;
    turn = process;

    while (turn == process && interested[other] == TRUE)
        ; /* busy wait */
}

void leave_region(int process)
{
    interested[process] = FALSE;
}
```

Answer 24

Peterson's solution works with preemptive scheduling. In fact, it was designed for that kind of situation, where a process can be interrupted at almost any point.

With nonpreemptive scheduling, busy waiting can cause a progress problem. If a process is spinning in a waiting loop, it may never give up the CPU, so the other process may never get a chance to run and change the condition being waited for.

For example, in the simpler **turn**-only solution, if **turn** is initially 0 but process 1 runs first, process 1 loops forever waiting for **turn** to become 1. Since scheduling is nonpreemptive, process 0 may never run to change **turn**.

Thus, preemptive scheduling is suitable for this kind of spin-based mutual-exclusion algorithm, while purely nonpreemptive scheduling can make busy waiting dangerous.

Question 25

Can the priority inversion problem discussed in Sec. 2.3.4 happen with user-level threads? Why or why not?

Answer 25

In the usual pure user-level thread system, this priority inversion problem does not occur in the same way.

Priority inversion happens when a low-priority thread or process is in its critical region, then a high-priority one becomes ready and preempts it. If the high-priority one busy waits for a resource held by the low-priority one, the low-priority one may never run again

to release the resource.

With user-level threads, the kernel does not schedule the individual threads. It schedules only the process that contains them. A low-priority user-level thread is not suddenly preempted by the kernel so that a high-priority user-level thread from the same process can run.

With kernel-level threads, however, the kernel schedules threads individually, so this kind of priority inversion can occur.

Question 26

In Sec. 2.3.4, a situation with a high-priority process, H , and a low-priority process, L , was described, which led to H looping forever. Does the same problem occur if round-robin scheduling is used instead of priority scheduling? Discuss.

Answer 26

With round-robin scheduling, the same permanent problem does not occur.

Under priority scheduling, the high-priority process H can keep running and busy waiting, while the low-priority process L never gets the CPU. Since L never runs, it never leaves its critical region, so H waits forever.

Under round-robin scheduling, every runnable process gets a time slice periodically. Sooner or later, L will run again. When it runs, it can leave its critical region and release the resource. After that, H can enter.

Thus, round-robin scheduling may still waste CPU time because of busy waiting, but it avoids the permanent starvation caused by strict priority scheduling.

Question 27

In a system with threads, is there one stack per thread or one stack per process when user-level threads are used? What about when kernel-level threads are used? Explain.

Answer 27

There is one stack per thread in both cases.

Each thread has its own sequence of procedure calls, local variables, parameters, and return addresses. These must be stored separately, so every thread needs its own stack.

This is true for user-level threads and kernel-level threads. With kernel-level threads, the operating system also commonly keeps a separate kernel stack for each thread, so that system calls and interrupts can be handled safely for that particular thread.

Question 28

When a computer is being developed, it is usually first simulated by a program that runs one instruction at a time. Even multiprocessors are simulated strictly sequentially like this. Is it possible for a race condition to occur when there are no simultaneous events like this?

Answer 28

Yes. A race condition does not require true physical simultaneity. It only requires an unlucky interleaving of operations on shared data.

For example, suppose process A reads a shared variable. Then a simulated clock tick

occurs and process *B* runs. Process *B* reads the same variable, adds 1, and stores it back. Later, process *A* resumes and also adds 1 based on the old value it read earlier. Even though the simulator executed one instruction at a time, the final value is wrong because the two simulated processes raced on the shared variable.

Question 29

The producer-consumer problem can be extended to a system with multiple producers and consumers that write to, or read from, one shared buffer. Assume that each producer and consumer runs in its own thread. Will the solution presented in Fig. 2-28, using semaphores, work for this system?

```
#define N 100          /* number of slots in the buffer */
typedef int semaphore; /* semaphores are a special kind of int */

semaphore mutex = 1;   /* controls access to critical region */
semaphore empty = N;   /* counts empty buffer slots */
semaphore full = 0;    /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

Answer 29

Yes. The same semaphore solution works with multiple producers and multiple consumers. The semaphore `mutex` ensures that only one thread at a time can access the shared buffer data structure. Therefore, two producers cannot insert at the same time, two consumers cannot remove at the same time, and a producer and consumer cannot corrupt the buffer by updating it concurrently.

The semaphore **empty** counts the number of empty slots, so producers block when the buffer is full. The semaphore **full** counts the number of filled slots, so consumers block when the buffer is empty.

Thus, the solution scales from one producer and one consumer to multiple producers and consumers.

Question 30

Consider the following solution to the mutual-exclusion problem involving two processes P_0 and P_1 . Assume that the variable **turn** is initialized to 0. Process P_0 's code is presented below.

```
/* Other code */
while (turn != 0) {
    /* Do nothing and wait. */
}

/* Critical section */
turn = 0;
/* Other code */
```

For process P_1 , replace 0 by 1 in the above code. Determine if the solution meets all the required conditions for a correct mutual-exclusion solution.

Answer 30

It satisfies mutual exclusion, but it is not a correct general solution.

As written, when **turn** is 0, only P_0 can enter its critical section. When **turn** is 1, only P_1 can enter. Therefore, both processes cannot be in the critical section at the same time. However, the solution fails the progress requirement. With the code exactly as shown, P_0 sets **turn** back to 0 when it leaves, and P_1 sets **turn** back to 1 when it leaves. Since **turn** starts at 0, P_1 may never get a chance to enter at all.

Even in the usual strict-alternation version, where each process gives the turn to the other process after leaving, the solution is still undesirable. A process can be forced to wait for the other process even when the other process is not interested in entering the critical section.

So the solution gives mutual exclusion, but it does not meet all the requirements for a correct mutual-exclusion algorithm.

Question 31

How could an operating system that can disable interrupts implement semaphores?

Answer 31

The operating system can make each semaphore operation atomic by disabling interrupts while it manipulates the semaphore.

For a **down** operation, the kernel disables interrupts, checks the semaphore value, and either decrements it or blocks the calling process on the semaphore's waiting list if the value is zero.

For an **up** operation, the kernel disables interrupts, checks whether any processes are blocked on the semaphore, and wakes one of them if necessary. If no process is waiting, it increments the semaphore value.

After the semaphore data structures have been updated, the kernel enables interrupts again. Since no interrupt can occur during the update, no other process can observe the semaphore in an inconsistent state.

Question 32

Show how counting semaphores, that is, semaphores that can hold an arbitrary value, can be implemented using only binary semaphores and ordinary machine instructions.

Answer 32

Each counting semaphore can be represented by:

- an integer counter,
- a binary semaphore M for mutual exclusion,
- a binary semaphore B used to block and wake waiting processes,
- a waiting list of processes blocked on the counting semaphore.

The **down** operation first does **down**(M) to get exclusive access to the counter and waiting list. It decrements the counter. If the counter is still nonnegative, it releases M and returns. If the counter is negative, the process is put on the waiting list, M is released, and the process blocks on B .

The **up** operation also starts with **down**(M). It increments the counter. If the counter is positive, no process is waiting, so it simply releases M . If the counter is zero or negative, some process is waiting. One process is removed from the waiting list and awakened using B ; then M is released.

The important point is that M protects the internal counter and waiting list, while B provides the actual blocking and wakeup mechanism.

Question 33

If a system has only two processes, does it make sense to use a barrier to synchronize them? Why or why not?

Answer 33

Yes. A barrier can make sense even with only two processes.

If the program runs in phases, and neither process is allowed to enter the next phase until both processes have completed the current phase, then a barrier is appropriate. Each process waits at the barrier until the other one arrives.

With only two processes, the barrier is simple, but the idea is still useful.

Question 34

Can two threads in the same process synchronize using a kernel semaphore if the threads are implemented by the kernel? What if they are implemented in user space? Assume that no threads in any other processes have access to the semaphore. Discuss your answers.

Answer 34

With kernel-level threads, yes. If one thread blocks on a kernel semaphore, the kernel knows that only that thread is blocked. It can still schedule another thread from the same process, including the thread that may eventually release the semaphore.

With pure user-level threads, this does not work well. If one user-level thread makes a blocking kernel semaphore call, the kernel sees only the whole process, not the individual user-level thread. The kernel may block the entire process. Then no other user-level thread in that process can run to release the semaphore.

So kernel semaphores are suitable for kernel-level threads, but they can break a pure user-level thread package if semaphore operations block in the kernel.

Question 35

Synchronization within monitors uses condition variables and two special operations, **wait** and **signal**. A more general form of synchronization would be to have a single primitive, **waituntil**, that had an arbitrary Boolean predicate as parameter. Thus, one could say:

```
waituntil x < 0 or y + z < n
```

The **signal** primitive would no longer be needed. This scheme is clearly more general than that of Hoare or Brinch Hansen, but it is not used. Why not?

Answer 35

It is not used because it is too expensive to implement efficiently.

With arbitrary predicates, the run-time system would have to know when each predicate might have changed from false to true. That means whenever any variable appearing in any waiting predicate changes, the system may have to re-evaluate the predicates of waiting processes.

This is much more expensive than ordinary condition variables. With condition variables, a process wakes up only when another process explicitly executes **signal**. The programmer tells the monitor exactly when a condition may have become true.

Thus, **waituntil** is more general, but its implementation cost is too high.

Question 36

A fast-food restaurant has four kinds of employees: order takers, who take customers' orders; cooks, who prepare the food; packaging specialists, who stuff the food into bags; and cashiers, who give the bags to customers and take their money. Each employee can be regarded as a communicating sequential process. What form of interprocess communication do they use? Relate this model to processes in UNIX.

Answer 36

They communicate by message passing.

The order takers pass orders to the cooks. The cooks pass prepared food to the packaging specialists. The packaging specialists pass bags to the cashiers. Each stage receives a message-like object, works on it, and passes it to the next stage.

In UNIX terms, this resembles a pipeline of processes connected by pipes. The output from one process becomes the input to the next process.

Question 37

Suppose that we have a message-passing system using mailboxes. When sending to a full mailbox or trying to receive from an empty one, a process does not block. Instead, it gets an error code back. The process responds to the error code by just trying again, over and over, until it succeeds. Does this scheme lead to race conditions?

Answer 37

No. This scheme does not lead to race conditions, assuming the mailbox send and receive operations themselves are atomic.

Nothing is lost: a failed send or receive simply returns an error code, and the process tries again later. The mailbox state is still protected by the message-passing system.

However, the scheme is inefficient because it is busy waiting. A process can waste CPU time repeatedly checking a mailbox that is still full or still empty.

Question 38

The CDC 6600 computers could handle up to 10 I/O processes simultaneously using an interesting form of round-robin scheduling called processor sharing. A process switch occurred after each instruction, so instruction 1 came from process 1, instruction 2 came from process 2, and so on. The process switching was done by special hardware, and the overhead was zero. If a process needed T sec to complete in the absence of competition, how much time would it need if processor sharing was used with n processes?

Answer 38

It would need:

$$\boxed{nT \text{ sec}}$$

With n processes sharing the processor instruction by instruction, each process effectively gets only $\frac{1}{n}$ of the processor. Since switching overhead is zero, the only slowdown is this sharing factor.

Question 39

Consider the following piece of C code:

```
void main(void)
{
    fork();
    fork();
    exit();
}
```

How many child processes are created upon execution of this program?

Answer 39

Three child processes are created.

Initially, there is one process. After the first `fork`, there are two processes: the original parent and one child.

Both of these processes then execute the second `fork`, so each creates one more child.

That adds two more child processes.
Thus, the total number of child processes created is:

$$1 + 2 = \boxed{3}$$

Question 40

Round-robin schedulers normally maintain a list of all runnable processes, with each process occurring exactly once in the list. What would happen if a process occurred twice in the list? Can you think of any reason for allowing this?

Answer 40

If a process occurred twice in the runnable list, it would receive two time quanta per full pass through the list. In effect, it would get a larger share of the CPU than processes that appear only once.

This could be used as a simple weighted round-robin scheme. More important processes could be inserted multiple times to give them more CPU time.

However, when such a process blocks or exits, all of its entries must be removed from the runnable list. Otherwise, the scheduler might later try to run a process that is no longer runnable.

Question 41

Can a measure of whether a process is likely to be CPU bound or I/O bound be determined by analyzing source code? How can this be determined at run time?

Answer 41

Sometimes source code gives a useful hint, but it is not always reliable.

For example, a program that reads all its input into memory at the start and then computes for a long time is likely to be CPU bound. A program that repeatedly reads and writes many files, such as a compiler, is more likely to be I/O bound.

At run time, the operating system can measure this more directly by comparing CPU time with elapsed wall-clock time. If a process uses CPU time close to its total elapsed time, it is CPU bound. If it uses much less CPU time than elapsed time, it probably spent much of its time waiting for I/O.

On UNIX-like systems, tools such as `ps` or `time` can help report CPU usage and elapsed time.

Question 42

Explain how time quantum value and context switching time affect each other in a round-robin scheduling algorithm.

Answer 42

The time quantum should be large compared with the context switching time.

If context switching takes a large amount of time and the quantum is small, the system wastes too much CPU time switching between processes instead of running them. In that case, the quantum should be increased.

However, if the quantum is too large, round-robin scheduling starts to behave more like

first-come, first-served scheduling. Interactive response time may become worse. If context switching time is very small, the scheduler has more freedom to use a smaller quantum without wasting too much CPU time on overhead.

Question 43

Measurements of a certain system have shown that the average process runs for a time T before blocking on I/O. A process switch requires a time S , which is effectively wasted overhead. For round-robin scheduling with quantum Q , give a formula for the CPU efficiency for each of the following:

1. $Q = \infty$
2. $Q > T$
3. $S < Q < T$
4. $Q = S$
5. Q nearly 0

Answer 43

CPU efficiency is:

$$\text{Efficiency} = \frac{\text{useful CPU time}}{\text{useful CPU time} + \text{switching overhead}}$$

If $Q \geq T$, the process blocks on I/O before its quantum expires. The cycle is T useful time plus S switching overhead, so:

$$\frac{T}{T + S}$$

This applies to:

$$Q = \infty$$

and:

$$Q > T$$

If $S < Q < T$, the process is preempted every Q time units. A run of length T needs about $\frac{T}{Q}$ process switches, wasting:

$$\frac{ST}{Q}$$

So the efficiency is:

$$\frac{T}{T + \frac{ST}{Q}} = \frac{Q}{Q + S}$$

For $Q = S$:

$$\frac{Q}{Q + S} = \frac{S}{S + S} = \boxed{\frac{1}{2} \text{ or } 50\%}$$

As $Q \rightarrow 0$:

Efficiency $\rightarrow 0$

Question 44

Five jobs are waiting to be run. Their expected run times are 9, 6, 3, 5, and X . In what order should they be run to minimize average response time? Your answer will depend on X .

Answer 44

To minimize average response time, use shortest job first. The jobs should be run in increasing order of expected running time.

$0 < X \leq 3$: $X, 3, 5, 6, 9$
 $3 < X \leq 5$: $3, X, 5, 6, 9$
 $5 < X \leq 6$: $3, 5, X, 6, 9$
 $6 < X \leq 9$: $3, 5, 6, X, 9$
 $X > 9$: $3, 5, 6, 9, X$

Question 45

Five batch jobs, A through E, arrive at a computer center at almost the same time. They have estimated running times of 10, 6, 2, 4, and 8 minutes. Their externally determined priorities are 3, 5, 2, 1, and 4, respectively, with 5 being the highest priority. For each of the following scheduling algorithms, determine the mean process turnaround time. Ignore process switching overhead.

- (a) Round robin.
- (b) Priority scheduling.
- (c) First-come, first-served, run in order 10, 6, 2, 4, 8.
- (d) Shortest job first.

For round robin, assume that the system is multiprogrammed and that each job gets its fair share of the CPU. For priority scheduling, first-come, first-served, and shortest job first, assume that only one job at a time runs until it finishes. All jobs are completely CPU bound.

Answer 45

The jobs are:

Job	A	B	C	D	E
Run time	10	6	2	4	8
Priority	3	5	2	1	4

For round-robin scheduling, the finishing times are:

10, 18, 24, 28, 30

So the mean turnaround time is:

$$\frac{10 + 18 + 24 + 28 + 30}{5} = \boxed{22 \text{ minutes}}$$

For priority scheduling, the order is B, E, A, C, D . The finishing times are:

6, 14, 24, 26, 30

So the mean turnaround time is:

$$\frac{6 + 14 + 24 + 26 + 30}{5} = \boxed{20 \text{ minutes}}$$

For first-come, first-served scheduling in the order A, B, C, D, E , the finishing times are:

10, 16, 18, 22, 30

So the mean turnaround time is:

$$\frac{10 + 16 + 18 + 22 + 30}{5} = \boxed{19.2 \text{ minutes}}$$

For shortest job first, the order is C, D, B, E, A . The finishing times are:

2, 6, 12, 20, 30

So the mean turnaround time is:

$$\frac{2 + 6 + 12 + 20 + 30}{5} = \boxed{14 \text{ minutes}}$$

Question 46

A process running on CTSS needs 30 quanta to complete. How many times must it be swapped in, including the very first time before it has run at all?

Answer 46

It must be swapped in:

$\boxed{5 \text{ times}}$

The first time it is swapped in, it gets 1 quantum. On later runs it gets larger allocations:

1, 2, 4, 8, 15

These add up to:

$$1 + 2 + 4 + 8 + 15 = 30$$

So the process needs five swaps in total, including the first one.

Question 47

Consider a real-time system with two voice calls of periodicity 5 msec each with CPU time per call of 1 msec, and one video stream of periodicity 33 msec with CPU time per call of 11 msec. Is this system schedulable?

Answer 47

Yes. The system is schedulable because the total CPU utilization is less than 1.
Each voice call runs:

$$\frac{1000}{5} = 200$$

times per second. Each call needs 1 msec of CPU time, so one voice call uses:

$$200 \times 1 = 200 \text{ msec/sec}$$

Two voice calls use:

$$400 \text{ msec/sec}$$

The video stream runs:

$$\frac{1000}{33} \approx 30.30$$

times per second. Each video activation needs 11 msec, so it uses:

$$30.30 \times 11 \approx 333.33 \text{ msec/sec}$$

Total CPU demand is:

$$400 + 333.33 = 733.33 \text{ msec/sec}$$

Since this is less than 1000 msec/sec, the system is schedulable.

Question 48

For the above problem, can another video stream be added and have the system still be schedulable?

Answer 48

No. Another video stream would add another:

$$333.33 \text{ msec/sec}$$

of CPU demand.

The new total would be:

$$400 + 333.33 + 333.33 = 1066.66 \text{ msec/sec}$$

This is more than 1000 msec/sec, so the system would not be schedulable.

Question 49

The aging algorithm with $a = \frac{1}{2}$ is being used to predict run times. The previous four runs, from oldest to most recent, are 40, 20, 40, and 15 msec. What is the prediction of the next time?

Answer 49

Using aging with $a = \frac{1}{2}$, each new prediction is the average of the previous prediction and the most recent actual run.

Starting with the oldest run as the first prediction:

$$P_1 = 40$$

After the next actual run, 20:

$$P_2 = \frac{1}{2}(40) + \frac{1}{2}(20) = 30$$

After the next actual run, 40:

$$P_3 = \frac{1}{2}(30) + \frac{1}{2}(40) = 35$$

After the most recent actual run, 15:

$$P_4 = \frac{1}{2}(35) + \frac{1}{2}(15) = 25$$

So the prediction for the next run is:

25 msec

Question 50

A soft real-time system has four periodic events with periods of 50, 100, 200, and 250 msec each. Suppose that the four events require 35, 20, 10, and x msec of CPU time, respectively. What is the largest value of x for which the system is schedulable?

Answer 50

For schedulability, the total CPU utilization must not exceed 1:

$$\frac{35}{50} + \frac{20}{100} + \frac{10}{200} + \frac{x}{250} \leq 1$$

Compute the known utilization:

$$0.70 + 0.20 + 0.05 = 0.95$$

So:

$$0.95 + \frac{x}{250} \leq 1$$

$$\frac{x}{250} \leq 0.05$$

$$x \leq 12.5$$

Thus, the largest value is:

12.5 msec

Question 51

In the dining philosophers problem, let the following protocol be used: An even-numbered philosopher always picks up the left fork before picking up the right fork; an odd-numbered philosopher always picks up the right fork before picking up the left fork. Will this protocol guarantee deadlock-free operation?

Answer 51

Yes. This protocol guarantees deadlock-free operation.

The reason is that not all philosophers try to acquire forks in the same direction. The circular wait condition is broken. There will always be at least one fork free, and at least one philosopher will be able to obtain both forks and eat.

After that philosopher finishes and releases both forks, progress continues. Therefore, the system cannot enter a state in which every philosopher is permanently waiting for the next fork.

Question 52

A real-time system needs to handle two voice calls that each run every 6 msec and consume 1 msec of CPU time per burst, plus one video at 25 frames/sec, with each frame requiring 20 msec of CPU time. Is this system schedulable?

Answer 52

Yes. The system is schedulable.

Each voice call runs:

$$\frac{1000}{6} \approx 166.67$$

times per second. Since each burst takes 1 msec, one voice call uses:

$$166.67 \text{ msec/sec}$$

Two voice calls use:

$$333.33 \text{ msec/sec}$$

The video stream runs 25 times per second, with each frame requiring 20 msec:

$$25 \times 20 = 500 \text{ msec/sec}$$

Total CPU demand is:

$$333.33 + 500 = 833.33 \text{ msec/sec}$$

Since this is less than 1000 msec/sec, the system is schedulable.

Question 53

Consider a system in which it is desired to separate policy and mechanism for the scheduling of kernel threads. Propose a means of achieving this goal.

Answer 53

One way is to let the kernel provide the scheduling mechanism, while allowing each process to define the scheduling policy for its own threads.

For example, the kernel can schedule processes using its normal scheduling algorithm. Within a process, the kernel can run that process's kernel threads strictly according to thread priorities. The user process is then allowed to assign or change the priorities of its own threads.

In this design, the kernel supplies the mechanism for context switching and dispatching threads, while the user process controls the policy by choosing the relative priorities of its threads.

Question 54

In the solution to the dining philosophers problem in Fig. 2-47, why is the state variable set to `HUNGRY` in the procedure `take_forks`?

Answer 54

The state is set to `HUNGRY` so that neighboring philosophers can later see that this philosopher wants to eat.

If the philosopher cannot get both forks immediately, the philosopher blocks. Later, when a neighbor puts down forks, the procedure `test` checks the blocked philosopher's state. If the state is `HUNGRY` and both forks are available, that philosopher can be awakened.

Without recording the `HUNGRY` state, the system would not know which blocked philosophers are waiting for forks.

Question 55

Consider the procedure `put_forks` in Fig. 2-47. Suppose that the variable `state[i]` was set to `THINKING` after the two calls to `test`, rather than before. How would this change affect the solution?

Answer 55

The solution would break.

When a philosopher finishes eating, the philosopher must first change state to `THINKING`. Only then should the philosopher test whether the left and right neighbors can eat.

If `state[i]` is still `EATING` during the two calls to `test`, the neighbors will appear to have an eating neighbor, so neither one will be allowed to start eating. This can leave hungry neighbors blocked even though the forks are actually available.

Thus, setting the state to `THINKING` before calling `test` is essential for waking eligible neighbors.

Question 56

The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring or not favoring some category of processes. For each variation, specify what happens when a reader or a writer becomes ready to access the database, and what happens when a process is finished.

Answer 56

One variation gives priority to readers.

In this version, a reader may start immediately unless a writer is currently active. If a writer is waiting but readers are active, new readers may still enter. When a writer finishes, all waiting readers are started before any waiting writer. This favors readers but can starve writers.

A second variation gives priority to writers.

In this version, no new reader may start if a writer is waiting. When the last active process finishes, a waiting writer is started if one exists. If no writer is waiting, all waiting readers may start. This prevents writer starvation but can delay readers.

A third variation is more symmetric.

When reading has started, new readers may continue to join while readers are active. When the last reader finishes, a waiting writer is started. When writing has started, waiting writers may be given priority until no writers remain; then readers may proceed. This groups access by type and tries to avoid switching back and forth too often.

Question 57

Write a shell script that produces a file of sequential numbers by reading the last number in the file, adding 1 to it, and then appending it to the file. Run one instance of the script in the background and one in the foreground, each accessing the same file. How long does it take before a race condition manifests itself? What is the critical region? Modify the script to prevent the race. Hint: use `ln file file.lock` to lock the data file.

Answer 57

There is no fixed time at which the race condition must appear. It depends on the timing of the two shell processes and the operating system scheduler. In practice, it often appears quickly when two copies run at the same time.

An unsafe version is:

```
#!/bin/sh

if [ ! -f numbers ]; then
    echo 0 > numbers
fi

count=0
while [ "$count" -ne 200 ]
do
    count=`expr "$count" + 1`
    n=`tail -1 numbers`
    expr "$n" + 1 >> numbers
done
```

Run one copy in the background and one in the foreground:

```
sh numbers.sh &
sh numbers.sh
```

The critical region is the part that reads the last number, adds 1, and appends the new number:

```
n=`tail -1 numbers`
expr "$n" + 1 >> numbers
```

If two processes read the same last number before either one appends the next number, both can append the same value or otherwise disturb the sequence. A simple locked version using `ln` is:

```
#!/bin/sh

if [ ! -f numbers ]; then
    echo 0 > numbers
fi

count=0
while [ "$count" -ne 200 ]
do
    if ln numbers numbers.lock 2>/dev/null
    then
        count=`expr "$count" + 1`
        n=`tail -1 numbers`
        expr "$n" + 1 >> numbers
        rm -f numbers.lock
    fi
done
```

The hard link succeeds only if the lock name does not already exist. While the lock exists, another copy skips that attempt. This protects the read-modify-append sequence from concurrent access.

Question 58

Assume that you have an operating system that provides semaphores. Implement a message system. Write the procedures for sending and receiving messages.

Answer 58

A bounded mailbox can be implemented with the same idea as the producer-consumer solution. Use one semaphore to count empty slots, one to count full slots, and one mutex semaphore to protect the mailbox data structure.

```
#define N 100

typedef struct message message;
message mailbox[N];
int in = 0;
int out = 0;

semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void send(message m)
{
    down(&empty);
    down(&mutex);

    mailbox[in] = m;
    in = (in + 1) % N;
```

```

    up(&mutex);
    up(&full);
}

message receive(void)
{
    message m;

    down(&full);
    down(&mutex);

    m = mailbox[out];
    out = (out + 1) % N;

    up(&mutex);
    up(&empty);

    return m;
}

```

If the mailbox is full, **send** blocks on **empty**. If the mailbox is empty, **receive** blocks on **full**. The semaphore **mutex** ensures that only one process changes the mailbox indices at a time.

Question 59

Solve the dining philosophers problem using monitors instead of semaphores.

Answer 59

A monitor solution keeps each philosopher's state inside the monitor and uses one condition variable per philosopher.

```

monitor DiningPhilosophers {
    enum { THINKING, HUNGRY, EATING } state[N];
    condition self[N];

    int left(int i) { return (i + N - 1) % N; }
    int right(int i) { return (i + 1) % N; }

    void test(int i)
    {
        if (state[i] == HUNGRY &&
            state[left(i)] != EATING &&
            state[right(i)] != EATING) {
            state[i] = EATING;
            signal(self[i]);
        }
    }

    void take_forks(int i)
    {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            wait(self[i]);
    }
}

```

```

    }

    void put_forks(int i)
    {
        state[i] = THINKING;
        test(left(i));
        test(right(i));
    }
}

```

The monitor gives mutual exclusion automatically: only one philosopher at a time can execute inside it. A philosopher blocks on its own condition variable if both forks are not available.

Question 60

A university changes its bathroom policy so that a bathroom can be in one of three states: empty, women present, or men present. When a woman is in a bathroom, other women may enter but no men may enter, and vice versa. Write procedures `woman_wants_to_enter`, `man_wants_to_enter`, `woman_leaves`, and `man_leaves`. You may use whatever counters and synchronization techniques you like.

Answer 60

One monitor-style solution is to keep counts of the people currently inside and waiting. The variable `last` is used to avoid always favoring the same group when the bathroom becomes empty.

```

monitor Bathroom {
    enum { NONE, WOMEN, MEN } mode = NONE;
    enum { NO_ONE, LAST_WOMEN, LAST_MEN } last = NO_ONE;

    int women = 0, men = 0;
    int waiting_women = 0, waiting_men = 0;

    condition women_q, men_q;

    void woman_wants_to_enter(void)
    {
        waiting_women++;
        while (mode == MEN ||
              (mode == NONE && waiting_men > 0 && last == LAST_WOMEN))
            wait(women_q);

        waiting_women--;
        women++;
        mode = WOMEN;
        signal(women_q);
    }

    void man_wants_to_enter(void)
    {
        waiting_men++;
        while (mode == WOMEN ||
              (mode == NONE && waiting_women > 0 && last == LAST_MEN))
            wait(men_q);
    }
}

```



```

        waiting_men--;
        men++;
        mode = MEN;
        signal(men_q);
    }

    void woman_leaves(void)
    {
        women--;
        if (women == 0) {
            mode = NONE;
            last = LAST_WOMEN;
            if (waiting_men > 0)
                broadcast(men_q);
            else
                broadcast(women_q);
        }
    }

    void man_leaves(void)
    {
        men--;
        if (men == 0) {
            mode = NONE;
            last = LAST_MEN;
            if (waiting_women > 0)
                broadcast(women_q);
            else
                broadcast(men_q);
        }
    }
}

```

The `while` loops are important because a process that wakes up must recheck whether the bathroom state still allows it to enter.

Question 61

Rewrite the program of Fig. 2-23 to handle more than two processes.

Answer 61

The strict-alternation idea can be generalized by letting `turn` range from 0 to $N - 1$.

```

#define N 10

int turn = 0;

void process(int i)
{
    while (TRUE) {
        while (turn != i)
            ; /* busy wait */

        critical_region(i);
    }
}

```

```

        turn = (turn + 1) % N;

        noncritical_region(i);
    }
}

```

This preserves mutual exclusion because only the process whose number equals **turn** can enter its critical region.

However, it has the same weakness as the two-process version. It enforces strict alternation. If process *i* is not interested in entering its critical region, all later processes may still be forced to wait for it to take and pass its turn.

Question 62

Write a producer-consumer problem that uses threads and shares a common buffer. However, do not use semaphores or any other synchronization primitives to guard the shared data structures. Just let each thread access them when it wants to. Use **sleep** and **wakeup** to handle the full and empty conditions. See how long it takes for a fatal race condition to occur.

Answer 62

The following is intentionally unsafe. It shows why the shared variables must be protected.

```

#define N 100

int buffer[N];
int in = 0;
int out = 0;
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();

        if (count == N)
            sleep(PRODUCER);

        buffer[in] = item;
        in = (in + 1) % N;
        count++;

        if (count == 1)
            wakeup(CONSUMER);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0)

```

```

        sleep(CONSUMER);

        item = buffer[out];
        out = (out + 1) % N;
        count--;

        if (count == N - 1)
            wakeup(PRODUCER);

        consume_item(item);
    }
}

```

The fatal race can occur at any time. The critical region is every access to the shared variables `buffer`, `in`, `out`, and `count`. Without mutual exclusion, the producer and consumer can update `count` inconsistently or suffer a lost wakeup.

Question 63

A process can be put into a round-robin queue more than once to give it a higher priority. Running multiple instances of a program each working on a different part of a data pool can have the same effect. First write a program that tests a list of numbers for primality. Then devise a method to allow multiple instances of the program to run at once in such a way that no two instances of the program will work on the same number. Can you in fact get through the list faster by running multiple copies of the program?

Answer 63

The key idea is to make claiming work atomic. Keep the untested numbers in a shared work file and use a lock file before removing the next number.

```

while true
do
    while ! ln work.txt work.lock 2>/dev/null
    do
        sleep 1
    done

    number='head -1 work.txt'
    if [ -z "$number" ]; then
        rm -f work.lock
        exit 0
    fi

    tail -n +2 work.txt > work.tmp
    mv work.tmp work.txt
    rm -f work.lock

    test_number_for_primality "$number"
done

```

The lock protects the operation that removes one number from the shared work pool. Once a process has claimed a number, it releases the lock and tests the number independently. Running multiple copies can finish the list faster if the machine has multiple CPUs or if the system is busy and the extra processes get a larger share of CPU scheduling opportunities.

On a single-user machine with one CPU and no competing processes, multiple CPU-bound copies usually do not make the work finish faster; they mostly divide the same CPU among themselves.

Question 64

Implement a multithreaded solution to determine whether a given number N is a perfect number. The main program reads N and P from the command line, spawns P threads, partitions the search space among them, stores factors in a shared buffer using appropriate synchronization, waits for all threads to finish, and then reports whether N is perfect.

Answer 64

A practical solution gives each thread a disjoint range of possible factors and protects the shared factor buffer with a mutex.

```
long N;
int P;

long factors[MAX_FACTORS];
int nfactors = 0;
pthread_mutex_t factor_lock = PTHREAD_MUTEX_INITIALIZER;

void add_factor(long x)
{
    pthread_mutex_lock(&factor_lock);
    factors[nfactors++] = x;
    pthread_mutex_unlock(&factor_lock);
}

void *worker(void *arg)
{
    long id = (long) arg;
    long limit = (long) sqrt((double) N);
    long start = id * limit / P + 1;
    long end = (id + 1) * limit / P;

    for (long i = start; i <= end; i++) {
        if (N % i == 0) {
            long other = N / i;

            if (i != N)
                add_factor(i);
            if (other != i && other != N)
                add_factor(other);
        }
    }

    return NULL;
}
```

The main thread creates P workers, joins them all, and then computes:

$$\text{sum} = \sum \text{all collected factors}$$

If:

$$\text{sum} = N$$

then N is perfect; otherwise it is not. Searching only up to $\sqrt{N}$ is sufficient because factors occur in pairs.

Question 65

Implement a program to count the frequency of words in a text file. The text file is partitioned into N segments. Each segment is processed by a separate thread that outputs the intermediate frequency count for its segment. The main process waits until all the threads complete; then it computes the consolidated word-frequency data based on the individual threads' output.

Answer 65

One clean design is to give each thread its own private word-frequency table. This avoids locking during the main counting work.

```
main(file, N):
    split file into N byte ranges
    adjust each range to start and end on word boundaries

    for each range i:
        create thread count_words(i)

    for each thread:
        join thread

    global_table = empty map
    for each thread_table:
        for each (word, count) in thread_table:
            global_table[word] += count

    print global_table

count_words(i):
    local_table = empty map
    for each word in assigned range i:
        normalize word
        local_table[word]++
    return local_table
```

The important detail is word-boundary handling. If the file is split purely by byte offset, a word may be cut in half. Each segment should therefore be adjusted so that only one thread counts each complete word.